

Software Architecture and Design Specification

Collaborative Study Material & Quiz Platform

Project: Collaborative Study Material & Quiz Platform

Version: 1.0

Authors: Rakshitha, Mokshith, Manasi, Mahesh

Date: 02-10-2026

Status: Draft

Revision history

Version	Date	Author	Change summary	Approval
1.0	02-10-2026	Rakshitha, Mokshith, Manasi, Mahesh	Initial SAD draft with component and sequence diagrams	

Approvals

Role	Name	Signature / Date
Course Coordinator		
Project Guide		

Team & Roles

Scrum role	Name	Technical ownership	Responsibility
Scrum Master	Rakshitha	—	Facilitates standups, maintains the sprint board, removes blockers, coordinates schedule
Product Owner	Mokshith	—	Prioritises backlog against the SRS, approves acceptance criteria, keeps documentation in sync with the build
Developer	Manasi	Integration testing	Designs & executes integration test cases across modules; verifies cross-module interactions
Developer	Mahesh	Frontend & CI/CD	Implements the frontend UI; sets up and maintains the CI/CD pipeline

Table of Contents

1. Introduction

2. Document Overview

3. Architecture

4. Design

5. Appendices

1. Introduction

1.1 Purpose

This document specifies the software architecture and design of the Collaborative Study Material & Quiz Platform, translating the requirements in the companion Software Requirements Specification (SRS) into a concrete system structure, component responsibilities, interaction flows, and technology choices.

1.2 Scope

Covers the platform's core services: user authentication, study material management (upload, search, versioning, plagiarism screening), quiz authoring and auto-graded quiz attempts, discussion and rating, leaderboard/analytics and notifications, and administrative moderation.

1.3 Audience

Developers, QA engineers, security auditors, instructors/course staff, and maintenance teams.

1.4 Definitions

SRS, STP, RTM, UI, API, JWT, RBAC, TLS, OTP, LMS, SLA, WCAG, CDN, ADR.

2. Document overview

2.1 How to use this document

Provides architectural deliverables including a UML component diagram, UML sequence diagrams, architecture rationale, a lightweight STRIDE threat model, and API design for the platform's core services.

2.2 Related documents

Software Requirements Specification (SRS) v1.0, Software Test Plan (STP) v1.0, Requirements Traceability Matrix (RTM).

3. Architecture

3.1 Goals & Constraints

Goals: scale to thousands of concurrent students during exam periods, secure handling of credentials and quiz content, a maintainable and independently deployable set of services, and 99.5% monthly availability (CSQ-NF-002).

Constraints: dependency on external Plagiarism Detection and Notification services, WCAG 2.1 AA accessibility (CSQ-NF-005), 2-year audit log retention (CSQ-NF-004), and a cloud budget typical of an academic/early-stage deployment.

3.2 Stakeholders & Concerns

- Students: fast, fair quizzes; a responsive material library
- Instructors: easy quiz/material authoring; reliable class analytics
- Administrators: effective moderation tools and a clear audit trail
- Developers: modular, testable, independently deployable services

3.3 Component (UML) Diagram

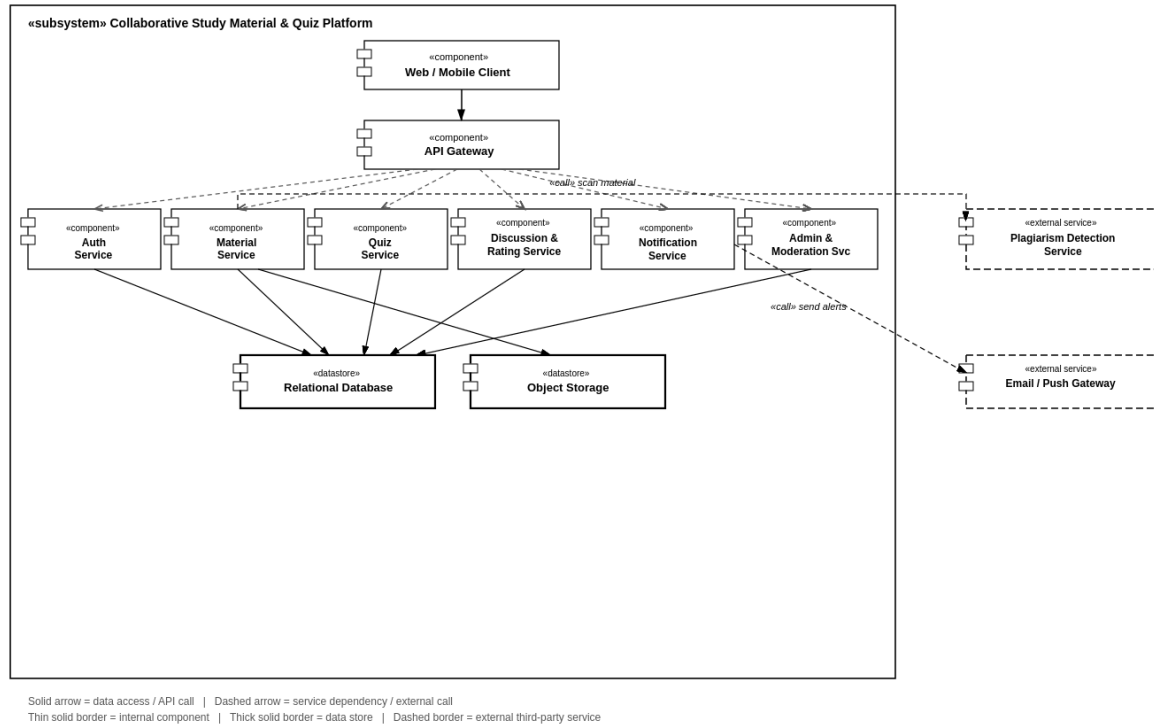


Figure 3.3 — UML component diagram for the Collaborative Study Material & Quiz Platform.

3.4 Component Descriptions

- Web / Mobile Client: responsive UI used by students, instructors, and administrators.
- API Gateway: single entry point; validates session tokens, routes requests, applies rate limiting.
- Auth Service: handles registration, login, JWT issuance, and account lockout (CSQ-F-001..003).
- Material Service: handles upload, tagging, versioning, search indexing, and triggers plagiarism scans (CSQ-F-010..013).
- Quiz Service: quiz authoring, question bank, timed attempts, and auto-grading (CSQ-F-020..032).
- Discussion & Rating Service: comments, threaded replies, and material ratings (CSQ-F-040..042).
- Notification Service: dispatches email/push notifications via the external gateway (CSQ-F-052).
- Admin & Moderation Service: user management, moderation queue, and audit logging (CSQ-F-060..062).
- Relational Database: stores users, quizzes, attempts, comments, ratings, and audit logs.
- Object Storage: stores uploaded files (PDFs, slides, documents).
- Plagiarism Detection Service (external) and Email/Push Gateway (external): third-party services integrated via the Material and Notification services respectively.

3.5 Chosen Architecture Pattern and Rationale

A microservices-oriented layered architecture (API Gateway fronting independently deployable services) was chosen so that quiz-taking traffic spikes (e.g. during exam week) can be scaled independently of material upload or discussion workloads. A single monolith was rejected because quiz-attempt bursts would compete with material/browsing traffic for the same resources. A fully serverless/function-per-endpoint design was also rejected due to cold-start latency concerns during timed quiz attempts.

3.6 Technology Stack & Data Stores

- Frontend: responsive web app (React) and a companion mobile client

- Backend: containerised microservices (Node.js / Spring Boot) behind an API Gateway
- Relational Database: PostgreSQL for users, quizzes, attempts, comments, ratings, audit logs
- Object Storage: S3-compatible storage for uploaded study material
- Cache: Redis for leaderboard and session data
- Transport security: TLS 1.2+ everywhere; deployment via Docker/Kubernetes with autoscaling

3.7 Risks & Mitigations

Risk	Mitigation
Plagiarism Detection Service outage blocks material publishing	Queue scans asynchronously; allow provisional publish pending manual review
Quiz traffic spikes during exam week degrade response time	Autoscale Quiz Service; cache question banks; pre-warm instances before scheduled deadlines
Leakage or tampering of quiz answers	Encrypt quiz content at rest (CSQ-SR-003); RBAC on Quiz Service endpoints
Notification Service / Email-Push Gateway downtime delays alerts	Retry with backoff; fall back to in-app notification center

3.8 Traceability to Requirements

Requirement	Component
CSQ-F-001 / CSQ-F-002 (Register & Login)	Auth Service
CSQ-F-010 / CSQ-F-011 (Upload & plagiarism check)	Material Service + Plagiarism Detection Service
CSQ-F-030 / CSQ-F-031 (Take quiz & auto-grade)	Quiz Service
CSQ-F-040 (Comment on material)	Discussion & Rating Service
CSQ-F-052 (Deadline / result notifications)	Notification Service
CSQ-F-060 / CSQ-F-061 (Suspend user / moderation queue)	Admin & Moderation Service

3.9 Security Architecture

Threat Modeling (STRIDE):

- Spoofing → JWT-based session tokens issued only after successful Auth Service validation; optional MFA for instructors/admins
- Tampering → TLS 1.2+ in transit; encryption at rest for quiz content (CSQ-SR-003)
- Repudiation → tamper-evident audit logging of all administrative actions (CSQ-SR-005)
- Information Disclosure → TLS everywhere; hashed passwords (CSQ-SR-002); least-privilege data access
- Denial of Service → API Gateway rate limiting and autoscaling of stateless services

- Elevation of Privilege → role-based access control (RBAC) enforced at the API Gateway and each service (CSQ-SR-004)

4. Design

4.1 Design Overview

The platform is designed as a set of layered, independently deployable services behind a single API Gateway. Each service owns its slice of the relational schema, communicates over REST, and publishes domain events (e.g. quiz graded, material published) consumed by the Notification Service.

4.2 UML Sequence Diagrams

Two representative flows are modelled below: a student taking a quiz, and an instructor uploading study material (including the plagiarism-check path).

Sequence Diagram 1 — Student Takes a Quiz

Collaborative Study Material & Quiz Platform

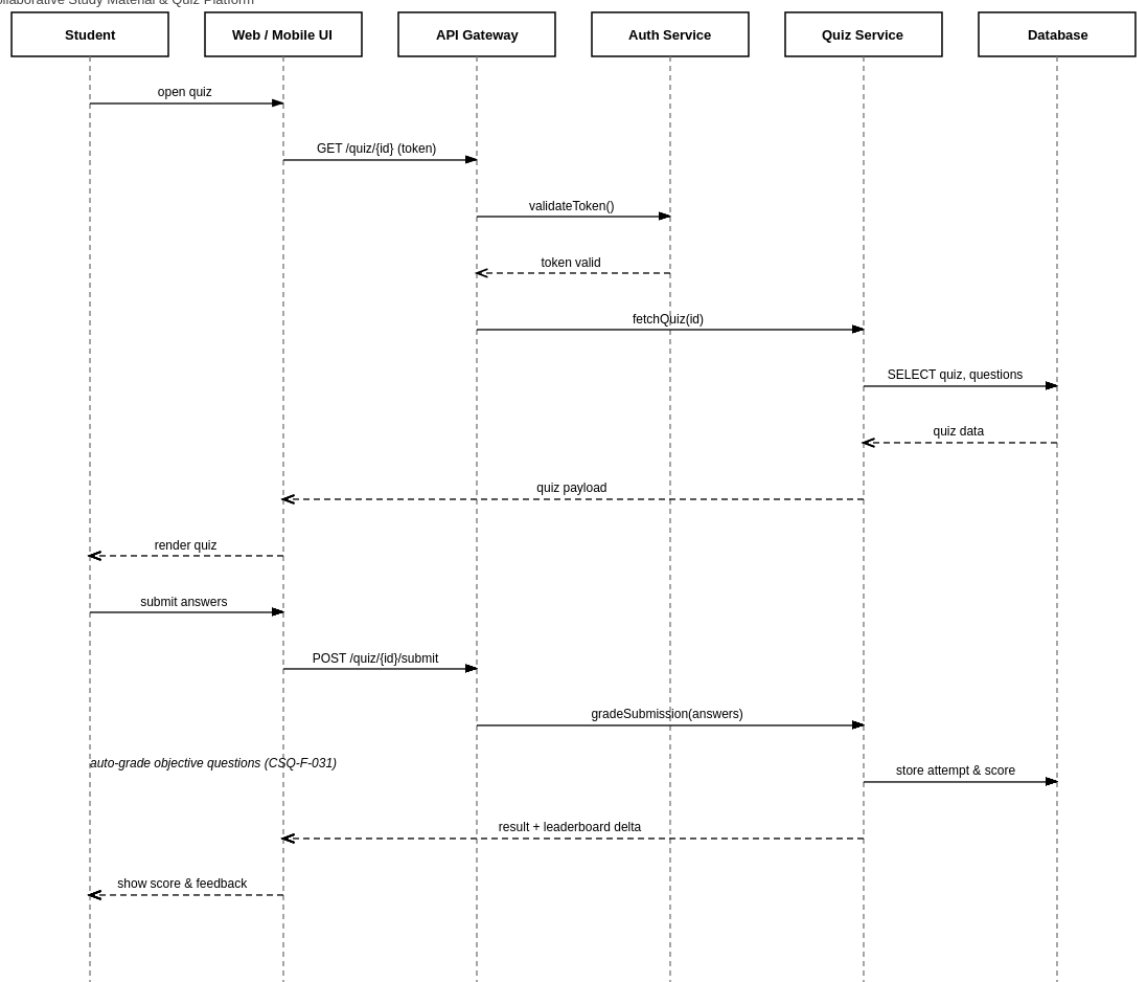


Figure 4.2a — Sequence diagram: Student takes a quiz.

Sequence Diagram 2 — Instructor Uploads Study Material
Collaborative Study Material & Quiz Platform

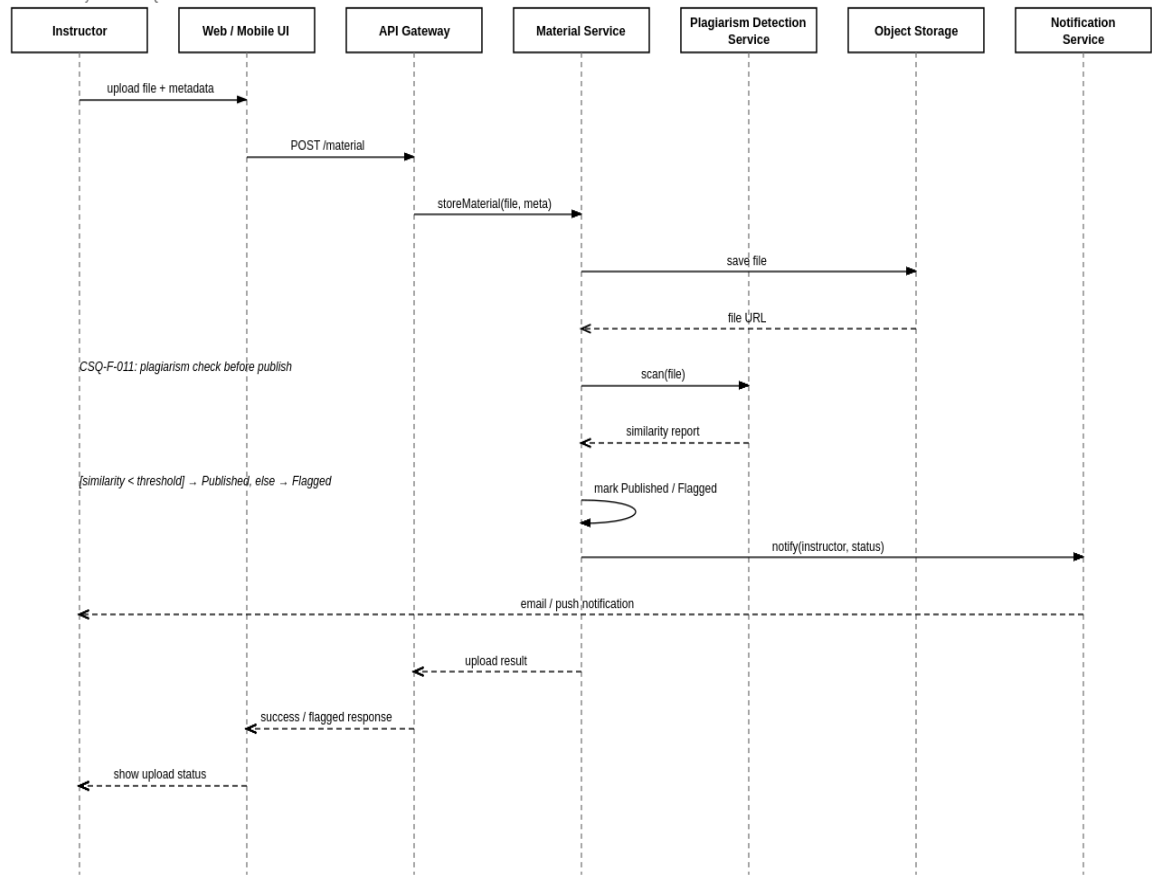


Figure 4.2b — Sequence diagram: Instructor uploads study material.

4.3 API Design

Representative endpoints for the Auth, Material, and Quiz services:

Endpoint	/api/v1/auth/login
Method	POST
Request	{email, password}
Response	{token, expiresIn}
Errors	401 Invalid credentials; 423 Account locked (CSQ-F-003)

Endpoint	/api/v1/materials
Method	POST
Request	{title, subject, topic, difficulty, file or link, tags[]}
Response	{materialId, status} (status = SCANNING PUBLISHED FLAGGED)
Errors	400 Invalid metadata; 403 Not an instructor; 422 Plagiarism threshold exceeded (CSQ-F-011)

Endpoint	/api/v1/quizzes/{quizId}/attempts
Method	POST
Request	{ } (student identified by JWT)
Response	{attemptId, questions[], endsAt}
Errors	403 Quiz not active or not published to this group; 409 Attempt limit exceeded (CSQ-F-021, CSQ-F-030)

Endpoint	/api/v1/quizzes/{quizId}/submit
Method	POST
Request	{attemptId, answers[]}
Response	{score, correctAnswers (if reveal enabled), leaderboardRank}
Errors	400 Malformed answers; 403 Quiz not active; 408 Time limit exceeded (CSQ-F-030..032)

4.4 Error Handling, Logging & Monitoring

Standardized error response format ({code, message}) across all services. No credentials, tokens, or quiz answers are written to logs. Monitoring covers quiz submission latency, plagiarism scan turnaround time, API error rate, and platform uptime, aligned to CSQ-NF-001 and CSQ-NF-002.

4.5 UX Design

Accessible colour contrast and font sizing, full keyboard navigation, a mobile-responsive quiz player with a visible countdown timer, and support for a high-contrast theme, consistent with CSQ-NF-005 (WCAG 2.1 AA).

4.6 Open Issues & Next Steps

- AI-assisted quiz question generation from uploaded material
- Offline quiz attempt caching for intermittent connectivity
- Tuning the plagiarism similarity threshold based on pilot data

5. Appendices

5.1 Glossary

SRS, STP, RTM, JWT, RBAC, TLS, WCAG, CDN, ADR, STRIDE.

5.2 References

IEEE 42010 (architecture description), OWASP ASVS, WCAG 2.1, NIST SP 800-63 (digital identity guidelines).

5.3 Tools

PlantUML / draw.io (diagrams), Figma (UX), Postman / Swagger-OpenAPI (API design & testing).